

OSSP (Operating Systems And Systems Programming) - 25CS2104E

K.Rishita(2520030367)

1.)Develop a C program using fork() that creates a parent and child process. Display the Process ID (PID), Parent Process ID (PPID), and process states at different stages of execution.

Design an experiment to observe process state transitions (Ready, Running, Waiting, Terminated) using Linux monitoring tools such as ps, top, and /proc. Document the observations.

```
#include<stdio.h>

#include<stdlib.h>

#include<unistd.h>

#include<sys/wait.h>


int main()
{
pid_t pid;


printf("Parent Process Started\n");
printf("Parent PID: %d\n",getpid());


pid=fork();


if(pid<0)
{
perror("fork failed");
exit(1);
}


if(pid==0)
```

```
{
printf("\nChild Process\n");
printf("Child PID: %d\n",getpid());
printf("Parent PID: %d\n",getppid());
printf("Child is running...\n");
sleep(10);
printf("Child process terminating...\n");
exit(0);
}
else
{
printf("\nParent Process\n");
printf("Parent PID: %d\n",getpid());
printf("Child PID: %d\n",pid);
printf("Parent is waiting for child...\n");
wait(NULL);
printf("Child terminated.\n");
printf("Parent process terminating.\n");
}

return 0;
}
```

OUTPUT:

```
rishita@LAPTOP-Q85I28SL:~$ nano week3.c
rishita@LAPTOP-Q85I28SL:~$ gcc week3.c
rishita@LAPTOP-Q85I28SL:~$ ./a.out
Parent Process Started
Parent PID: 561

Parent Process
Parent PID: 561
Child PID: 562
Parent is waiting for child...

Child Process
Child PID: 562
Parent PID: 561
Child is running...
Child process terminating...
Child terminated.
Parent process terminating.
rishita@LAPTOP-Q85I28SL:~$ |
```